

Technical Assessment - Backend

Backend Developer Technical Assessment

Task 1: User Data Handling

Objective:

Develop a backend service that efficiently handles user data and supports basic CRUD operations. This service should be modular, scalable, and secure, ensuring best practices in handling data and API development.

Task Requirements

1. Main Features:

- Develop a RESTful API that performs CRUD operations for managing **User Profiles**. Each profile should have the following fields:
 - ID (auto-generated)
 - Name (string, required)
 - Email (string, unique, required)
 - Age (integer, optional)
 - CreatedAt (timestamp, auto-generated)
 - Set up database models to handle the above fields. Use **MongoDB** as the database.
 - Add token-based authentication (e.g., JWT) for the endpoints.
 - Implement the following API endpoints:
 - **POST** `/users`: Create a new user profile.
 - **GET** `/users`: Fetch all user profiles (support pagination and optional filtering by age).
 - **GET** `/users/:id`: Fetch a specific user profile by ID.
 - **PUT** `/users/:id`: Update an existing user profile.
 - **DELETE** `/users/:id`: Delete a user profile by ID.
-

2. Assessment Guidelines:

- **Task Breakdown:** Include comments or documentation that explicitly outline how you decompose the task into smaller subtasks and prioritize them.
 - **Data Validation:** Implement validation for required fields and ensure the uniqueness of the email field.
 - **Error Handling:** Handle errors gracefully, including invalid input, missing resources, and server errors.
 - **Modular Code Structure:** Use a clean folder structure, such as separating controllers, models, routes, and utilities.
 - **Performance Optimization:** Use indexes where necessary and ensure efficient database queries.
 - **Security:** Sanitize inputs and ensure no SQL injection or other vulnerabilities are possible.
-

3. Bonus Points:

- **Deployment:** Deploy the API to a service like **Heroku**, **Render**, or **AWS** and provide the live URL.
 - **Documentation:** Use a tool like **Swagger** or **Postman** to create API documentation with examples.
 - **Testing:** Write unit and integration tests for critical parts of the application.
-

Task 2: Handling Delivery Slots

Objective:

Using Pseudocode, design the logic for a backend system of a **customer-facing app** where users can select delivery slots for their orders. The backend will handle the logic for assigning delivery slots dynamically, ensuring no overbooking, balancing slot capacity and providing real-time feedback to customers.

Problem Context

Customers use an app to place orders and select preferred delivery slots. Each slot has a limited capacity, and the backend system must:

1. Process incoming booking requests.

2. Assign slots based on availability.
3. Suggest alternative slots if the preferred slot is unavailable.

The backend system handles this logic and ensures the app provides customers with immediate responses.

Example Flow in the App

1. Customer opens the app and selects an order.
2. The app displays all slots, available slots are clickable while fully booked slots are disabled.
3. The customer selects a clickable slot and submits it.
4. The app shows feedback for the booking process.

The **pseudocode** you provide should guide the backend implementation.

Pseudocode Syntax: Find the Largest Number in a List

Below are two examples for writing your pseudocode.

Simple Pseudocode

This version provides a high-level overview of the solution, focusing on the main steps without getting into implementation specifics.

```
function findLargestNumber(numbers):  
  - Set the first number as the largest.  
  - Loop through the numbers in the list.  
  - If a number is larger than the current largest, update the largest.  
  - Return the largest number.
```

Explanation:

This pseudocode outlines the key logic in a concise manner without going into detailed syntax or error handling. It demonstrates the general thought process but lacks finer implementation specifics.

Detailed Pseudocode

This version provides a step-by-step breakdown, accounting for edge cases and detailed operations.

```

function findLargestNumber(numbers):
    if numbers is empty:
        return "Error: The list is empty"
    largest = numbers[0] # Assume the first number is the largest.

    for each number in numbers starting from the second:
        if number > largest:
            largest = number # Update largest if a bigger number is found.

    return largest # Return the largest number found.

# Example usage:
numbers = [12, 7, 23, 9]
result = findLargestNumber(numbers)
print("The largest number is:", result)

```

Explanation:

This version delves into more granular details, including:

- Error handling for an empty list.
 - Explicit initialization and update of the largest number.
 - A clear example of how the function would work in practice.
-

Candidate Task

Using these examples as a guide:

- Write pseudocode to solve the problem of **dynamic delivery slot allocation** for the back-end of the customer app.
- You may choose to present it as either **simple** or **detailed** pseudocode, however, **candidates submitting a Detailed Pseudocode will receive Bonus Points**.
- Place your pseudocode file at the root directory with the name and format of your choice.

Requirements for Pseudocode:

1. Allocate delivery slots dynamically based on availability.
2. Prevent overbooking by tracking the number of booked slots.
3. Suggest alternative slots if a customer's preferred slot is unavailable.
4. Delivery Slots are a shared resource; handle them accordingly.

Assessment Deliverables:

1. Provide the link to a public GitHub repository, or alternatively, a Drive Link for a ZIP file containing the source code by email to turuq.hr@gmail.com with the subject "Backend Assessment" .
 2. Include a README file that explains:
 - How to set up and run the project locally.
 - Optionally, any challenges faced and how you overcame them.
 3. If deployed, provide the live API URL and example requests.
-